```cpp
#include <iostream>
#include<vector>
using namespace std;
void Merge(vector<int> &arr ,int low,int high , int mid ){
    int left = low ;
    int right = mid +1;
    vector<int>temp;
    while(left<=mid && right<=high){
        if(arr[left]<=arr[right]){
            temp.push_back(arr[left]);
            left++;
        }
        if(arr[right]<=arr[left]){
            temp.push_back(arr[right]);
            right++;
        }
    }
    while(left<=right){
        temp.push_back(arr[left]);
        left++;
    }
    while(right<=high){
        temp.push_back(arr[right]);
        right++;
    }
    for(int i = low ; i<=high; i++){
        arr[i]=temp[i-low];

    }
```

```
6
450

1200
800
650
300
1500
300 450 650 800 1200 300

=== Code Execution Successful ===
```

```
}
void Mergesort(vector<int>&arr, int low , int high){
    if(low == high){
        return;
    }
    int mid = low+(high-low)/2;
    Mergesort(arr, low, mid);

    Mergesort(arr,mid+1, high);
    Merge(arr,low, high,mid);


}

int main() {
    int n ;
    cin>>n;
   vector<int> arr(n);
    for(int i = 0; i<n ; i++){
        cin>>arr[i];
    }
    Mergesort(arr,0,n-1);
    for(int i = 0 ; i<n ; i++){
        cout<<arr[i]<<" ";
    }



    return 0;
}
```

```
6
450

1200
800
650
300
1500
300 450 650 800 1200 300

=== Code Execution Successful ===
```

```cpp
#include <iostream>
#include<vector>
using namespace std;
void Merge(vector<int> &arr ,int low,int high , int mid ){
    int left = low ;
    int right = mid +1;
    vector<int>temp;
    while(left<=mid && right<=high){
        if(arr[left]<=arr[right]){
            temp.push_back(arr[left]);
            left++;
        }
        if(arr[right]<=arr[left]){
            temp.push_back(arr[right]);
            right++;
        }
    }
    while(left<=right){
        temp.push_back(arr[left]);
        left++;
    }
    while(right<=high){
        temp.push_back(arr[right]);
        right++;
    }
    for(int i = low ; i<=high; i++){
        arr[i]=temp[i-low];

    }
```

```
7
12
25
18
30
15
22
40
12 15 18 22 25 18 15
Median: 22
Orders Above Median: 1
Difference: 3


=== Code Execution Successful ===
```

```cpp
}
void Mergesort(vector<int>&arr, int low , int high){
    if(low == high){
        return;
    }
    int mid = low+(high-low)/2;
    Mergesort(arr, low, mid);

    Mergesort(arr,mid+1, high);
    Merge(arr,low, high,mid);


}

int main() {
    int n ;
    cin>>n;
   vector<int> arr(n);
    for(int i = 0; i<n ; i++){
        cin>>arr[i];
    }
    Mergesort(arr,0,n-1);
    for(int i = 0 ; i<n ; i++){
        cout<<arr[i]<<" ";
    }
    cout<<endl;
```

```
7
12
25
18
30
15
22
40
12 15 18 22 25 18 15
Median: 22
Orders Above Median: 1
Difference: 3


=== Code Execution Successful ===
```

```cpp
vector<int> arr(n);
 for(int i = 0; i<n ; i++){
     cin>>arr[i];
 }
 Mergesort(arr,0,n-1);
 for(int i = 0 ; i<n ; i++){
     cout<<arr[i]<<" ";
 }
 cout<<endl;

 int median;
 if (n % 2 == 1)
     median = arr[n/ 2];
 else
     median = (arr[n / 2 - 1] + arr[n/ 2]) / 2;

 cout << "Median: " << median << endl;

 int count = 0;
 for (int x : arr) {
     if (x > median)
         count++;
 }

 cout << "Orders Above Median: " << count << endl;

 cout << "Difference: " << arr[n - 1] - arr[0] << endl;

 return 0;
```

```
7
12
25
18
30
15
22
40
12 15 18 22 25 18 15
Median: 22
Orders Above Median: 1
Difference: 3


=== Code Execution Successful ===
```

```cpp
#include <iostream>
using namespace std;
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
    cin >> n;

    int arr[n];

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    quickSort(arr, 0, n- 1);

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

```
6
120
45
78
200
60
95
45 60 78 95 120 200

=== Code Execution Successful ===
```

```cpp
#include <iostream>
using namespace std;
int partition(long long arr[], int low, int high) {
    long long pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] > pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}
void quickSort(long long arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int N;
    cin >> N;

    long long arr[N];
    long long totalSum = 0;
```

```
8
500
1200
700
2000
900
1500
3000
2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3


=== Code Execution Successful ===
```

```cpp
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
        totalSum += arr[i];
    }
    double overallAvg = (double)totalSum / N;
    quickSort(arr, 0, N - 1);
    for (int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "Top 5: ";
    long long topSum = 0;
    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
        topSum += arr[i];
    }
    cout << endl;
    cout << "Average of Top 5: " << topSum / 5 << endl;
    int count = 0;
    for (int i = 0; i < N; i++) {
        if (arr[i] > overallAvg)
            count++;
    }

    cout << "Values Above Overall Average: " << count << endl;

    return 0;
}
```

```
8
500
1200
700
2000
900
1500
3000
2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3


=== Code Execution Successful ===
```

```cpp
#include <iostream>
using namespace std;
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int N;
    cin >> N;

    int arr[N];

    for (int i = 0; i < N; i++) {
        cin >> arr[i];
    }

    heapSort(arr, N);

    for (int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

```
6
850
1200
950
600
1500
1000
600 850 950 1000 1200 1500

=== Code Execution Successful ===
```

```cpp
#include <iostream>
#include <iomanip>
using namespace std;
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

```
8
12
7
15
9
20

5
10
8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%


=== Code Execution Successful ===
```

```cpp
int main() {
    int N;
    cin >> N;

    int arr[N];
    double sum = 0;
    for (int i = 0; i < N; i++) {
        cin >> arr[i];
        sum += arr[i];
    }
    heapSort(arr, N);
    for (int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "Fastest: " << arr[0] << endl;
    cout << "Slowest: " << arr[N - 1] << endl;
    double avg = sum / N;
    cout << "Average: " << fixed << setprecision(2) << avg << endl;
    int count = 0;
    for (int i = 0; i < N; i++) {
        if (arr[i] < avg)
            count++;
    }

    cout << "Cases Faster Than Average: " << count << endl;
    double percentage = (count * 100.0) / N;
    cout << "Percentage: " << fixed << setprecision(2) << percentage << "%" << endl;

    return 0;
}
```

```
8
12
7
15
9
20

5
10
8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%


=== Code Execution Successful ===
```

Build your business.
Let QuillBot build
Try QuillBot for free